



Computer Games Development
Project Report
Year IV

[Jad Fuhr]
[C00290965]
[12/04/2026]

Contents

Acknowledgements	2
Project Abstract	2
Project Introduction and/or Research Question.....	2
Literature Review	2
Evaluation and Discussion	3
Conclusions	3
References	4
Appendices	4

Acknowledgements

I would like to thank my lecturers, supervisor and peers at SETU for their guidance throughout this project.

I also acknowledge the creators of SFML and the nlohmann JSON library, whose tools formed the foundation of this project.

Project Abstract

This project presents the design and implementation of PaleoPals, a 2D mining and fossil-collection game built using C++ and SFML 3. The game features a destructible tile-based world, a player-controlled miner, collectible fossils that contribute to museum exhibits, and an upgrade and hiring system that drives in-game progression.

A central technical achievement of the project is the use of the nlohmann JSON library to drive all game data at runtime -including terrain layer definitions, building configurations, collectible types, and full dinosaur skeleton data. This data-driven approach decouples content from code, allowing the Museum Interior, Fossil Manager, and Trader systems to load, parse, and display their content entirely from external JSON files without recompilation.

A second key achievement would be my automated paleontologist npc, whose behaviour is based off a behaviour tree rather than a state machine. The NPC mines using a randomized depth first search, then navigating home with a breadth first search and finally retrieving the dropped collectibles with A*. Each search was chosen for a specific issue in mind rather than applying one algorithm. The npc deposits its findings through the same methods as player

A third key achievement is the upgrade system implemented through the Trader menu. Players can purchase incremental improvements to ray length, mining damage, pickup radius, and jump height, with costs calculated dynamically. This system integrates with the in-game economy and gives the player a meaningful progression loop throughout the experience.

The resulting game demonstrates how a well-structured data pipeline, an

automated NPC and a clear progression economy can produce an engaging gameplay experience. The work provides a strong foundation for future expansion including additional terrain layers, more dinosaur species, and a more complex upgrade tree.

Project Introduction and Research Question

Background

PaleoPals is a 2D mining and exploration game where the player digs through layered terrain to uncover fossils, amber, and other collectibles. These items contribute to museum displays, unlock cosmetic skins, and generate money used for upgrades. The world is fully destructible - each tile has hardness, hit points, and crack states, and breaking tiles may spawn collectibles. The player interacts with two surface buildings: the Museum, which displays collected dinosaur skeletons, and the Trader, which manages upgrades and palaeontologist hiring.

A core design goal was for all game content - terrain types, collectible definitions, dinosaur species and their skeleton pieces - to be defined externally in JSON rather than hardcoded. This required a robust data-loading pipeline capable of parsing nested JSON structures and mapping them to in-game objects at runtime.

Motivation

As the scope of the game grew, it became clear that a hardcoded approach to content would be unmanageable. Defining every fossil piece, terrain layer, and building position in C++ source code would make iteration slow and error prone. This motivated the shift to a fully data-driven architecture powered by `nlohmann::json`.

Similarly, giving the player meaningful choices required more than passive collection - an upgrade system tied to an in-game economy was needed to create a sense of progression and reward.

Research Question

How can a data-driven JSON pipeline, a behaviour tree driven automated agent, and a structured upgrade economy be combined to create a flexible and engaging 2D mining game?

Project Objectives

- Implement a JSON-driven data pipeline for terrain, collectibles, fossils, and dinosaur species.
- Implement an autonomous paleontologist NPC using a behaviour tree architecture rather than a state machine.
- Apply three distinct search algorithms, a depth-first, breadth-first, and A*, each selected for the navigation problem it best fits.

- Build a Museum Interior system that dynamically populates its displays from loaded JSON data.
- Create a Fossil Manager that spawns and tracks collectibles using runtime-loaded configuration.
- Design an upgrade system integrated with an in-game economy.
- Ensure all content is modifiable without recompiling the game.

Main Contributions

- A complete JSON data pipeline using `nlohmann::json` covering map layers, buildings, collectibles, and dinosaurs.
- A Museum Interior that reads fossil piece textures and dinosaur metadata directly from JSON and renders them dynamically.
- A Fossil Manager that uses JSON-defined spawn chances and piece assignments to generate collectibles at runtime.
- A Trader upgrade system with dynamic cost calculation integrated into the player's money economy.
- A destructible tile world whose hardness values, textures, and layer structure are all data-driven.
- A behaviour tree framework built on a single tick/BTStatus interface, supporting composite sequence and selector nodes alongside four leaf behaviours, with the NPC routine assembled from them at runtime.
- Three pathfinding implementations serving different roles: randomised DFS for excavation, BFS for shortest return trips across broken terrain, and A* with a Manhattan heuristic for collectible retrieval.

Literature Review

Data-Driven Game Design

Data-driven design is a widely adopted pattern in game development, separating content definition from game logic. By externalizing data to formats such as JSON or XML, developers can iterate content without recompilation and allow non-programmers to adjust game parameters. JSON is particularly well-suited to this because of its human-readable syntax and support for nested structures.

The `nlohmann JSON for Modern C++` library (`nlohmann::json`) is a popular choice in C++ game projects due to its intuitive API that mirrors STL containers. It supports structured access, range-based iteration, and type-safe

parsing, making it ideal for loading complex game configurations such as nested dinosaur piece definitions.

Behaviour Trees for Game AI

Behaviour trees emerged from robotics and commercial game AI as an alternative to finite state machines, which become difficult to maintain as the number of states grows and transitions multiply between them. Ögren's work on the topic sets out the core advantage: behaviour is composed from small, independent nodes that share one interface, so a routine can be restructured by changing how nodes are assembled rather than by editing transition logic spread across the system.

The three-value return convention, success, failure, and running, is what distinguishes the pattern from a simple decision tree. Running lets a node hold control across multiple frames, which suits game AI where most useful actions take time. Failure bubbles upward through composite nodes, so recovery behaviour is a property of the tree's structure rather than something each state must handle for itself.

Pathfinding in Grid-Based Worlds

Grid based games present navigation problems that differ enough to warrant different algorithms. BFS guarantees the shortest path on an unweighted graph and needs no heuristic, making it a natural fit where the graph is small and optimality matters. A*, as explained accessibly by Computerphile and by Natsu Games, extends this with a heuristic estimate of remaining distance, directing the search toward the goal instead of expanding evenly outward. On a 4 connected grid the Manhattan distance is admissible, so A* returns an optimal path while expanding far fewer nodes than BFS.

DFS is rarely used for navigation because it does not find shortest paths, but that same property makes it useful for generation. A DFS with randomised neighbour ordering produces long, winding corridors rather than compact regions, a technique commonly applied to maze generation and applicable here to excavation.

Upgrade and Progression Systems

Upgrade systems are a fundamental component of mining and idle games. Research on player motivation shows that incremental progression - where each purchase produces a visible, quantifiable improvement - drives continued engagement. Games such as Terraria, Minecraft Dungeons, and SteamWorld Dig use upgrade economies tied to resource collection as their primary progression loop.

Effective upgrade systems share common traits: transparency (the player can see what they are buying), immediacy (upgrades take effect right away), and meaningful impact (the difference is noticeable during gameplay). The PaleoPals Trader menu was designed with these principles in mind.

Museum and Collection Mechanics

Collection-completion mechanics, popularised by games such as Animal Crossing and Pokemon, give players a long-term goal that complements the short-term loop of resource gathering. Visually updating a display as items are collected provides positive feedback and a sense of achievement. Implementing this requires a data structure that tracks partial completion states per species and maps collected pieces to their visual representations.

Influence on this Project

- nlohmann::json was selected for its clean C++ integration and support for the nested data structures required by the fossil and dinosaur systems.
- Upgrade system design was informed by mining/idle game conventions emphasising clear cost-to-benefit ratios.
- The Museum Interior design was influenced by collection-completion mechanics that reward progressive discovery.
- Behaviour trees were chosen over a state machine for the paleontologist so the routine could be extended by composition, and so failure recovery could be handled structurally.
- Each of the three navigation problems was matched to the algorithm that suits its constraints, rather than applying a single general-purpose search.

Evaluation and Discussion

JSON Data Pipeline

The JSON data pipeline was evaluated across all systems that depend on it:

Map and Terrain

- Four terrain layers (Topsoil, Sediment, Rock, Bedrock) load correctly with their hardness values and texture paths.
- Building positions (Museum, Trader) are read from JSON and placed accurately in the world.
- Modifying hardness values in config.json produces immediate and correct changes in-game without recompilation.

Fossil Manager and Collectibles

- All collectible types (fossil fragments, amber, trash) load from JSON with correct textures, frame dimensions, and monetary values.
- Dinosaur species and their individual skeleton pieces (skull, torso, pelvis, tail, etc.) are fully defined in JSON and correctly assigned to spawned fossil collectibles at runtime.
- Spawn chance logic correctly interprets JSON-defined probability values.

Museum Interior

- Dinosaur skeleton displays populate correctly from loaded JSON data.
- Piece-by-piece collection state is tracked and reflected in the museum display accurately.
- Skin toggling reads alternate texture paths defined in JSON and switches correctly.

Upgrade System

- All four upgrades (ray length, damage, pickup radius, jump height) apply correctly to player stats immediately on purchase.
- Upgrade costs are calculated and displayed dynamically in the Trader UI.
- The money system correctly deducts purchase costs and reflects the current balance.

Paleontologist AI

Behaviour Trees

- The routine executes reliably across repeated cycles: surface patrol, excavation, return, collection, return.
- Running correctly holds multi-frame behaviours in place, with the sequence resuming at the active child rather than restarting each frame.
- Failure propagation resets the NPC to the top of the routine as intended, recovering from states that would otherwise strand it.

Pathfinding

- DFS excavation produces varied tunnel shapes across runs, confirming the randomised neighbour ordering has the intended effect.
- BFS consistently returns the shortest available route home across broken tiles.
- A* successfully retrieves scattered collectibles, and its heuristic keeps the cost acceptable despite the search running once per target.

Robustness

The most instructive part of the evaluation was how the AI failed rather than how it succeeded. Three distinct failure modes surfaced during testing, each needing a different fix:

- The NPC could finish excavating with no route home, leaving it stuck underground. Resolved by having the mining node fail the sequence and reset the tree.
- The NPC could target a collectible that A* could not reach and treat that single failure as evidence that no work remained. This was traced to two distinct conditions, first "this target failed" and secondly "nothing is left" which ended up in both being represented by a null target pointer. Separating them fixed this behaviour.
- Marking an unreachable collectible as permanently skipped proved too aggressive of a method, because the world changes as the NPC digs. Replacing the permanent skip with a couple attempts per target budget, cleared on every successful pickup, allowed genuinely reachable items to be retried.

I could only see each of these observably at runtime over repeated attempts rather than through purely the inspection of the code, which I would say is characteristic of an automated system where the AI/NPC reaches states the dev did not expect or plan for

Performance

- JSON loading occurs at startup only, no runtime parsing overhead during gameplay.
- View-frustum culling for collectibles ensures only visible items are drawn.
- No noticeable performance impact from the data-driven systems during normal play.

Limitations

- The JSON schema is not formally validated at load time, malformed config files produce undefined behaviour.
- Upgrade tiers are fixed at one level each, a multi-tier upgrade tree was not implemented.
- No save/load system, all progress and upgrades are lost on exit.
- Only one paleontologist is instantiated, the architecture supports more, but no claim system exists to stop two NPCs pathing to the same collectible.
- The Hire Paleontologist button is present in the Trader UI but is not wired to any behaviour in the current build.
- A* runs a fresh search for each collectible with no path caching, repeating most of its work between nearby targets.

Milestones

The original milestone plan included a Researcher AI role and a more complex sonar upgrade system. These were descoped in favour of delivering a complete implementation of the JSON data pipeline, Museum Interior, Fossil Manager, Trader upgrade system, and the paleontologist behaviour tree.

Key milestones achieved:

- November 30th: Destructible tile world and basic player movement implemented.
- December 31st: Fossil Manager and collectible spawning operational. Museum screen and fossil codex completed. Trader screen implemented.
- January 31st: Economy and upgrade system integrated. JSON pipeline extended to cover all content types.
- February 28th: Museum Interior fully populating from JSON data. Upgrade purchases applying correctly to player stats.
- March 31st: Code refactoring for clarity and efficiency. JSON schema stabilised.
- April 12th: Final polish, bug fixing, and documentation.
- August 2nd: Basic NPC DFS working
- August 4th: Implemented Ray Mining tool
- August 5th: pre behaviour tree dfs mining and beginnings of bfs returning works
- August 8th: Implemented fastest route back to surface
- August 13th: Implemented Beginnings of Behaviour Tree
- August 17th: implemented my root node and started leaf node headers (existing behaviours that now exist in the behaviour tree)
- August 18th: Separated node headers into respective cpp and header files
- August 19th: Begun the basic implementation of Astar Pathfinding
- August 21st: Basics of Astar Pathfinding working (looking good) but my NPC would teleport back to the starting position at the end (not looking good)
- August 23rd: fixed my teleporting NPC
- August 24th: hooked up the NPC gaining money and fossil pieces
- August 25th: Implemented the NPC depth upgrade
- August 28th: Finalized version of my Astar algorithm

Technical Achievement

- Implemented a complete runtime JSON data pipeline using nlohmann::json covering all game content.
- Designed and implemented a Museum Interior that dynamically renders fossil displays from JSON-defined species data.
- Built a Fossil Manager that spawns, assigns, and tracks collectibles entirely from external configuration.
- Created a Trader upgrade system with dynamic cost calculation and real-time stat application.
- Implemented a destructible tile world with four JSON-configured terrain layers.
- Ensured all content is fully data-driven and modifiable without code changes.
- Implemented a behaviour tree framework from scratch, including composite sequence and selector nodes and four leaf behaviours, with the NPC routine assembled by composition at startup.
- Implemented three search algorithms, randomised DFS, BFS, and A* with a Manhattan heuristic, each applied to the navigation problem it best suits.

Review

What Went Right

- The nlohmann::json pipeline made adding new dinosaur species and collectible types straightforward and fast.
- The Museum Interior provides strong visual feedback for collection progress.
- The upgrade system integrates cleanly with the economy and creates a satisfying progression loop.
- Separating content from code made iteration significantly faster throughout development.
- The behaviour tree made the NPC routine straightforward to reason about and extend, and its failure semantics handled recovery cases that would have needed explicit handling in a state machine.
- Routing NPC collection through the Player kept the money calculation and museum notification in a single place, so the AI required no changes to either system.

What Went Wrong

- Tile-to-world coordinate conversion required significant debugging time.
- Some UI elements needed rework due to camera zoom and fullscreen handling.
- No formal JSON schema validation means content errors are hard to diagnose.
- The AI was left until late in development, which compressed the time available for testing it. Several failure modes only appear after the NPC has run for minutes at a time and finding them under time pressure was not fun to say the least.

Outstanding Work

- A save/load system to persist upgrades and museum progress between sessions.
- Multi-tier upgrades with more granular cost curves.
- Procedural fossil distribution using weighted JSON-defined rarity values.
- More dinosaur species and collectible types added via JSON only.
- Multiple simultaneous paleontologists with a claim system that prevents targeting the same collectible.
- Specialised NPC roles composed from the existing behaviour tree node set.

If I Could Start Again

- Define the JSON schema formally before writing any parsing code.
- Plan UI scaling and camera behaviour earlier in development.
- Build the upgrade system earlier so it can influence overall game balance from the start.
- Build the AI earlier. It was implemented late and worked, but the failures in an automated agent only surface over long runs, and that testing time should have been accounted for rather than found out.

Tech Choices

- C++ and SFML: Good performance and control which is the correct choice for a 2D tile game.
- nlohmann::json: Excellent for C++ data-driven design, clean API, no dependencies, integrates with STL.
- JSON for configuration: Enables fast content iteration without recompilation.

Conclusion

This project demonstrates how a data-driven JSON pipeline, an automated agent, and a structured upgrade economy combine to form the technical backbone of a 2D mining game. The `nlohmann::json` library enabled all game content, terrain, collectibles, fossil species, and building data to be defined externally, producing a system that is flexible, and easy to iterate on.

The paleontologist NPC demonstrates the second half of the question. Its behaviour tree composes independent nodes into a routine that can be restructured without rewriting its transition logic, and its three search algorithms show the trade-offs between them applied to problems that genuinely differ: DFS for generating tunnels, BFS for guaranteed shortest returns, and A* for repeated searches against scattered goals.

Together with the Trader and Museum, these systems create an easy to follow progression loop: the player mines to earn money, spends it on upgrades that improve mining efficiency or extend the NPC's reach, and fills the museum as a long-term goal. The NPC depth upgrade closes the loop by converting money directly into passive income.

Future work would focus on a save/load system, multiple simultaneous paleontologists with a shared claim system, and procedural fossil distribution which are all natural extensions of my existing architecture.

References

Videos Used

Ögren, P. (2021). Behavior trees playlist [YouTube playlist]. YouTube.
https://www.youtube.com/playlist?list=PLFQdM4LOGDr_vYJuo8YTRcmv3FrwczdKg

Computerphile. (2014, December 18). A search algorithm* [Video]. YouTube.
<https://www.youtube.com/watch?v=ySN5Wnu88nE>

Natsu Games. (2023). A pathfinding explained simply* [Video]. YouTube.
<https://www.youtube.com/watch?v=pXsPj6HewhM>

Shah, M. (2020). C++ programming playlist [YouTube playlist]. YouTube.

<https://www.youtube.com/playlist?list=PLvv0ScY6vfd8j-tlhYVPYgiIyXduu6m-L>

Books Used

Meyers, S. (1996). More effective C++: 35 new ways to improve your programs and designs. Addison-Wesley.

Documentation Used

SFML Team. (2024). SFML 3.0.0 documentation. SFML. <https://www.sfml-dev.org/documentation/3.0.0/>

Websites Used

Nlohmann. (2024). basic_json API documentation. JSON for Modern C++. https://json.nlohmann.me/api/basic_json/

Some Quick Reads that Helped:

ScholarWorks@GVSU, “Behavior Trees in Robotics and AI,” Accessed: Aug. 30, 2026. [Online]. Available: <https://scholarworks.gvsu.edu/cgi/viewcontent.cgi?article=1367&context=cistechlib>

Lisyrus, “Behavior Trees,” Accessed: Aug. 30, 2026. [Online]. Available: <https://lisyrus.github.io/blog/posts/behavior-trees.html>